

Name : B. Ganga Jayadev Reddy

Reg. No: 192472248

Course Code & course Name: CSA0401 - Operating Systems

Faculty Name : Dr. J. Ruby Elizabeth

Question 1 – CPU Scheduling in a Real-Time Ticket Booking System

1. Most Suitable Scheduling Algorithm:

Priority Scheduling is the most appropriate algorithm because the ticket booking system contains tasks with different levels of importance. Payment gateway transactions are the most critical because a delay may cause payment failures or duplicate transactions. Therefore, these processes should always receive CPU time before less important tasks.

2. Justification:

- P1 (Payment Gateway) should have the highest priority because successful payment is essential before confirming a ticket.
- P2 (Seat Availability Check) should execute after payment so that seats can be reserved correctly.
- P3 (Ticket Confirmation and Printing) can execute last because it depends on the completion of the previous tasks.

Priority Scheduling improves CPU utilization by allowing the processor to execute the most important tasks first. It also reduces customer waiting time and improves the responsiveness of the booking system during heavy festival traffic.

3. Sample Execution Order:

High Priority : P1 – Payment Gateway

Medium Priority : P2 – Seat Availability Check

Low Priority : P3 – Ticket Confirmation & Printing

Execution Sequence:

P1 → P2 → P3

4. Drawback:

The major drawback of Priority Scheduling is starvation. If high-priority processes

continue to arrive, low-priority processes may never get CPU time. This problem can be reduced using an aging technique, where the priority of waiting processes gradually increases.

Question 2 – Deadlock in a Banking Transaction Server

1. Text-Based Resource Allocation Graph (RAG):

T1 holds Account Database (A) and requests Customer Database (B).

T2 holds Customer Database (B) and requests Transaction Logs (C).

T3 holds Transaction Logs (C) and requests Account Database (A).

2. Deadlock Analysis:

Yes, the system is in a deadlock because each transaction is waiting for a resource held by another transaction.

Circular Wait:

$T1 \rightarrow B \rightarrow T2 \rightarrow C \rightarrow T3 \rightarrow A \rightarrow T1$

Since none of the transactions can proceed, the system remains permanently blocked.

3. Recommended Technique – Deadlock Prevention:

Deadlock can be prevented by forcing every transaction to request resources in a fixed order ($A \rightarrow B \rightarrow C$). Since all transactions follow the same order, circular waiting cannot occur.

4. Locking Order Redesign:

Every transaction should first lock the Account Database, then the Customer Database, and finally the Transaction Logs. Releasing resources immediately after use further reduces waiting time and improves overall system performance.

Question 3 – Mobile OS App Switching Lag

1. Why CPU Remains Idle:

Although many applications are active, most are waiting for I/O operations such as reading data, accessing storage, or network communication. While these applications wait, the CPU has no useful work to perform. Frequent context switching also wastes processor time.

2. Time Quantum Analysis:

A Round Robin time quantum of 20 ms is not ideal for interactive mobile applications. Users experience delayed responses because foreground applications must wait longer before receiving CPU time.

3. Recommendation:

Use Priority-Based Round Robin or reduce the time quantum to about 10 ms. Foreground applications such as WhatsApp, phone calls, and music playback should receive higher priority than background applications.

4. Benefits:

The proposed scheduling method reduces waiting time, improves response time, decreases app switching lag, increases throughput, and provides a smoother user experience.

Question 4 – Deadlock Risk in an Operating System

Update

1. Can Deadlock Occur?

Yes. The given allocation satisfies the circular wait condition and can lead to a deadlock.

2. Circular Wait:

M1 holds R1 and waits for R2.

M2 holds R2 and waits for R3.

M3 holds R3 and waits for R1.

This forms the cycle:

$M1 \rightarrow R2 \rightarrow M2 \rightarrow R3 \rightarrow M3 \rightarrow R1 \rightarrow M1$.

3. Banker's Algorithm:

Before granting any resource request, the operating system checks whether the allocation leaves the system in a safe state. If granting the request leads to an unsafe state, the request is delayed.

A safe sequence is:

$M1 \rightarrow M2 \rightarrow M3 \rightarrow M4$.

Thus, deadlock is avoided.

4. Module Prioritization:

Priority Order:

1. M1 – Copy Update Files
2. M2 – Install Drivers
3. M3 – Update Registry
4. M4 – Validate System Integrity

Executing modules in this order minimizes resource conflicts, improves CPU utilization, and allows critical update tasks to finish earlier.

Question 5 – Cloud Server CPU Utilization Analysis

Given:

I/O wait probability (p) = 0.8

Number of Virtual Machines (n) = 7

Formula:

$$\text{CPU Utilization} = 1 - p^n$$

$$= 1 - (0.8)^7$$

$$= 1 - 0.2097$$

$$= 0.7903$$

$$\approx 79\%$$

1. CPU Utilization:

The CPU utilization is approximately 79%, indicating that the processor remains busy most of the time.

2. Interpretation:

A utilization of about 79% means the server is efficiently utilized. However, since VM processes spend significant time waiting for I/O, there is still scope for optimization.

3. Effect of Changing the Number of VMs:

Increasing the number of VMs generally improves throughput because more processes are available when others wait for I/O. However, too many VMs can increase memory usage and scheduling overhead. Reducing the number of VMs decreases throughput and leaves the CPU idle more often.

4. OS-Level Strategies:

- Use efficient CPU scheduling to reduce idle time.
- Apply load balancing so that VMs are evenly distributed across servers.
- Optimize I/O scheduling to reduce waiting time.
- Allocate resources dynamically based on workload demand.